

Block Drill Exam — Set 5 (hardest)

Ten blocks; each has exactly one real defect or none; the decoys are strong

October 2026

How to run it.

Same method. 12 minutes per block. Five of ten are clean. Write the verdict before uncovering; the point is calibration.

Block 1 — Async queue drain

```
1 class Drain {
2   private q: Array<() => Promise<void>> = [];
3   private running = false;
4   push(task: () => Promise<void>) { this.q.push(task); void this.run(); }
5   private async run() {
6     if (this.running) return;
7     this.running = true;
8     try {
9       while (this.q.length) await this.q.shift()!();
10    } finally {
11      this.running = false;
12    }
13  }
14 }
```

Requirement:

tasks run one at a time, in order, each awaited; a throwing task does not stop later tasks.

Notes

Line	Note
4	<code>void this.run()</code> : explicit fire-and-forget; <code>run</code> never rejects? see 9
6–7	guard then set, no await between: safe
9	<code>shift()!</code> justified by <code>length</code> ; each task awaited: sequential, in order
9	if a task THROWS, the <code>while</code> exits via the exception, <code>finally</code> resets <code>running</code> , and <code>run()</code> rejects: the <code>void</code> on line 4 means that rejection is unhandled, AND the remaining tasks in <code>q</code> are not run until the next <code>push</code>
10–11	<code>finally</code> resets the flag: good

Findings

- Line 9: a throwing task aborts the loop and leaves queued tasks stranded, plus an unhandled rejection via `void`; wrap the call: `try { await task() } catch (e) { report(e) }` inside the loop.

Confidence 3 · Request changes.

Everything else is right.

Block 2 — Sorted insert

```
1 function insertSorted(arr: number[], x: number): void {
2   let lo = 0, hi = arr.length;
3   while (lo < hi) {
4     const mid = (lo + hi) >>> 1;
5     if (arr[mid] < x) lo = mid + 1; else hi = mid;
6   }
7   arr.splice(lo, 0, x);
8 }
```

Requirement: insert `x` into an ascending array in place, keeping it sorted; stable for equal values (new value after existing equals).

Notes

Line	Note
2	<code>hi</code> exclusive
4	<code>>>> 1</code> is integer halving; no overflow for array lengths
5	<code>arr[mid] < x</code> → go right; equal → go left: finds the FIRST index with value $\geq x$: that places the new value BEFORE existing equals
7	<code>splice</code> mutates in place: required

Findings

1. Line 5: for stability “after existing equals” the comparison must be `<=` (upper bound), not `<`.

Confidence 4 · Request one change. Binary search is otherwise correct, including the `>>> 1`.

Block 3 — Token bucket

```
1 class Bucket {
2   private tokens: number;
3   private last: number;
4   constructor(private cap: number, private perSec: number, private now = Date.now) {
5     this.tokens = cap; this.last = now();
6   }
7   take(n = 1): boolean {
8     const t = this.now();
9     this.tokens = Math.min(this.cap, this.tokens + ((t - this.last) / 1000) * this.perSec);
10    this.last = t;
11    if (this.tokens < n) return false;
12    this.tokens -= n;
13    return true;
14  }
15 }
```

Requirement: classic token bucket; `take(n)` succeeds if `n` tokens available; refills at `perSec`; never exceeds `cap`.

Notes

Line	Note
4	injectable clock; <code>private now = Date.now</code> as a parameter property
9	refill proportional to elapsed; clamped by <code>cap</code> ; computed BEFORE the check: correct
10	<code>last</code> updated even on failure: correct (elapsed time was already credited)
11–12	boundary: <code>tokens < n</code> fails; <code>tokens === n</code> succeeds and leaves 0: correct
7	<code>n <= 0</code> or <code>NaN</code> : <code>tokens < NaN</code> false → "succeeds", <code>tokens -= NaN</code> → <code>NaN</code> forever (edge)

Findings

- Edge: validate `n` is a positive finite number.

Confidence 5 • Approve.

The float arithmetic here is intended; tokens are a rate, not money.

Block 4 — Deep get

```
1 function get(obj: unknown, path: string): unknown {  
2   return path.split(".").reduce((o, k) => (o as any)?.[k], obj);  
3 }
```

Requirement: `get(o, "a.b.c")` returns the nested value or `undefined` if any step is missing; must not throw.

Notes

Line	Note
2	<code>reduce</code> from <code>obj</code> ; each step <code>?.[k]</code> returns <code>undefined</code> on a nullish accumulator and keeps going; <code>undefined?.["x"]</code> is <code>undefined</code> ; never throws
2	<code>(o as any)</code> is the one cast; the function's contract is <code>unknown</code> in, <code>unknown</code> out, so the hole is contained
2	<code>path = ""</code> $\rightarrow$ <code>[""]</code> $\rightarrow$ <code>obj[""]</code> : <code>undefined</code> unless there is an empty-string key (edge, acceptable)
2	<code>"__proto__"</code> in the path reads the prototype (read-only, no pollution)

Findings

- None.

Confidence 5 · Approve. The `as any` is contained and necessary.

Block 5 — Semaphore with timeout

```
1 class Sem {
2   private free: number; private waiters: Array<() => void> = [];
3   constructor(n: number) { this.free = n; }
4   async acquire(timeoutMs: number): Promise<void> {
5     if (this.free > 0) { this.free--; return; }
6     await new Promise<void>((resolve, reject) => {
7       const t = setTimeout(() => reject(new Error("timeout")), timeoutMs);
8       this.waiters.push(() => { clearTimeout(t); resolve(); });
9     });
10  }
11  release() {
12    const w = this.waiters.shift();
13    if (w) w(); else this.free++;
14  }
15 }
```

Requirement: `acquire` gets a permit or rejects after `timeoutMs`; `release` hands the permit directly to the next waiter or returns it.

Notes

Line	Note
5	fast path, sync
7–8	timeout rejects; the waiter callback clears the timeout and resolves
13	permit handed directly to a waiter (no <code>free++</code>), so the count stays right: correct, and it is the fix for Set 3 Block 5
7 vs 12	if the timeout FIRES, the waiter's closure is still in <code>waiters</code> ; a later <code>release()</code> shifts it and calls it: <code>clearTimeout</code> on a fired timer is harmless, <code>resolve()</code> on a rejected Promise is ignored, BUT the permit was handed to a waiter that already gave up: the permit is LOST (neither <code>free++</code> nor a live acquirer)

Findings

- Lines 7/12: on timeout, remove the waiter from `waiters` (or mark it dead and have `release` skip dead waiters and return the permit); otherwise each timed-out waiter permanently consumes one permit.

Confidence 3 • Request changes.

Subtle; the hand-off pattern is otherwise correct.

Block 6 — Immutable update

```
1 type State = { users: Record<string, { name: string; tags: string[] }> };
2 function addTag(state: State, id: string, tag: string): State {
3   const user = state.users[id];
4   if (!user) return state;
5   return {
6     ...state,
7     users: { ...state.users, [id]: { ...user, tags: [...user.tags, tag] } },
8   };
9 }
```

Requirement:

return a new state with the tag appended; never mutate the input; unknown id returns the same state object.

Notes

Line	Note
4	returns the same reference for unknown id: required
5–8	new outer, new <code>users</code> , new user, new <code>tags</code> array: every level on the changed path is copied; other users remain shared (correct and intended for structural sharing)
3	<code>state.users[id]</code> with a user-controlled <code>id</code> of "constructor": <code>user</code> is a function, truthy, <code>user.tags</code> is <code>undefined</code> , <code>[...undefined]</code> THROWS

Findings

- 1. Line 3–4: `Record` on a plain object; `id = "constructor"` passes the `!user` check and throws at `[...user.tags]`; use `Object.hasOwn(state.users, id)` or a `Map`.

Confidence 4 • Request one change.

The immutable update itself is textbook.

Block 7 — Exponential moving average

```
1 function ema(values: number[], alpha: number): number[] {
2   if (values.length === 0) return [];
3   const out = [values[0]];
4   for (let i = 1; i < values.length; i++) out.push(alpha * values[i] + (1 - alpha) * out[i -
1]);
5   return out;
6 }
```

Requirement: EMA with smoothing `alpha` in (0, 1]; first output equals first input; output length equals input length.

Notes

Line	Note
2	empty handled
3	seed with first value
4	standard recurrence; <code>out[i - 1]</code> always exists; <code>i < length</code> correct
1	<code>alpha</code> not validated; <code>alpha = 0</code> returns a constant series; <code>alpha > 1</code> oscillates; <code>NaN</code> poisons (edge)

Findings

- Edge: validate `0 < alpha <= 1`.

Confidence 5 • Approve.

Block 8 — Dedupe concurrent requests

```
1  const pending = new Map<string, Promise<Response>>();
2  function dedupedFetch(url: string): Promise<Response> {
3    const p = pending.get(url) ?? fetch(url).finally(() => pending.delete(url));
4    pending.set(url, p);
5    return p;
6  }
```

Requirement: identical concurrent `fetch`s share one request; once settled, the next call fetches again.

Notes

Line	Note
3	<code>??</code> : reuse if present, else start and schedule removal on settle; <code>.finally</code> returns a new Promise that settles with the same outcome
4	re-sets the SAME <code>p</code> on a hit (harmless) or the new one on a miss
3	<code>.finally</code> deletes by <code>url</code> ; if a THIRD call arrived after the first settled but... no: the delete happens in the finally, and the next <code>get</code> after that misses: correct
3	<code>Response</code> body can be consumed once; two callers sharing one <code>Response</code> and both calling <code>.json()</code> → the second throws "body used already" (contract issue, not this function's bug; note it)

Findings

- Note: sharing a `Response` object means only one caller can read the body; callers must `.clone()` or the function should share the parsed result instead. Depending on the requirement wording (“share one request”) this is acceptable.

Confidence 4 • Approve with a note.

Block 9 — Windowed max

```
1 function windowMax(a: number[], k: number): number[] {
2   const out: number[] = [], dq: number[] = [];
3   for (let i = 0; i < a.length; i++) {
4     while (dq.length && a[dq[dq.length - 1]] <= a[i]) dq.pop();
5     dq.push(i);
6     if (dq[0] <= i - k) dq.shift();
7     if (i >= k - 1) out.push(a[dq[0]]);
8   }
9   return out;
10 }
```

Requirement: sliding-window maximum, window $k \geq 1$; output length $n - k + 1$.

Notes

Line	Note
2	<code>dq</code> holds indices, decreasing values
4	pop smaller-or-equal from the back: monotonic deque; <code><=</code> is fine (ties keep the newer index, which lasts longer)
6	evict the front if it left the window: <code>dq[0] <= i - k</code> is the correct boundary (index <code>i - k</code> is outside a window ending at <code>i</code>)
7	start emitting once the first full window exists: <code>i >= k - 1</code> : correct
1	<code>k > a.length</code> → empty output (correct: $n - k + 1 < 1$); <code>k <= 0</code> unvalidated

Findings

- Edge: validate `k >= 1`.

Confidence 5 · Approve. Every boundary here is right; check each one against `k = 1` and `k = n`.

Block 10 — Cancellable interval

```
1 function every(ms: number, fn: () => void): () => void {
2   let id: ReturnType<typeof setInterval> | undefined = setInterval(fn, ms);
3   return () => { if (id !== undefined) { clearInterval(id); id = undefined; } };
4 }
```

Requirement: run `fn` every `ms`; the returned function cancels; calling cancel twice is safe.

Notes

Line	Note
2	interval started; id captured
3	cancel clears once and nulls; second call is a no-op: idempotent
2	no <code>.unref()</code> : the interval keeps a Node process alive until cancelled; the requirement does not mention process exit (note, not finding)
1	<code>fn</code> that throws inside the interval: an uncaught exception in a timer callback crashes Node; the requirement does not say to handle it (note)

Findings

- None against the requirement.

Confidence 5 · Approve.

Scoring yourself

Block	Verdict	The one thing
1	request changes	a throwing task strands the queue
2	request one change	<code><</code> vs <code><=</code> for stability
3	approve	
4	approve	the <code>as any</code> is contained
5	request changes	timed-out waiter eats a permit
6	request one change	<code>"constructor"</code> as id
7	approve	
8	approve with note	shared <code>Response</code> body
9	approve	
10	approve	

Five approvals, four single-defect blocks, one note. If your count differs by more than one in either direction, redo the mismatched blocks with the card, out loud.